

Spring Boot — Lecture 8

XML-Based Configuration

1. Overview — Two Ways to Configure the IoC Container

The Spring IoC Container can receive its configuration metadata in two ways. Both produce the same result — bean creation, DI, lifecycle management. The container doesn't care which approach you use.

Approach	How you tell Spring what to manage	Class used in Java
Annotation-based (modern)	@Component, @Bean, @Configuration, @ComponentScan on classes	AnnotationConfigApplicationContext
XML-based (legacy)	beans.xml file in resources folder — tags	ClassPathXmlApplicationContext

XML-based configuration is used in legacy codebases. Spring Boot uses annotation-based only. Still worth knowing for legacy projects, migration work, and interviews.

2. Project Setup

Same dependency as before — spring-context. The only difference is the XML file location and the ApplicationContext class used.

```
org.springframework
spring-context
7.0.7
```

File location:

File	Location	Why
beans.xml	src/main/resources/beans.xml	Resources folder = classpath. ClassPathXmlApplicationContext reads from classpath.
AppConfig.java	src/main/java/.../AppConfig.java	Used in annotation-based approach — NOT needed for XML-based.

```
// Main.java — XML-based context setup
ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");

// Then use context.getBean() as usual
OrderService order = context.getBean("orderService", OrderService.class);
order.placeOrder();

context.close(); // triggers @PreDestroy / destroyMethod callbacks
```

3. beans.xml — File Structure

The XML file needs a header (schema metadata) at the top, then tags inside the root element.

The header just declares the XML schema (what tags are allowed). Copy it from the Spring documentation — nobody memorizes it.

4. Creating a Bean — Tag

A tag is equivalent to putting `@Component` on a class or writing a `@Bean` method in `AppConfig`.

XML attribute	Meaning	Required?
id	Unique identifier for this bean. Used to retrieve it via <code>getBean()</code> . Convention: camelCase version of class name.	Optional but strongly recommended
class	Fully qualified class name including package path. e.g. <code>in.strikcs.payment.UpiPaymentService</code>	Required
name	Alias(es) for the bean — can have multiple, space-separated. Both id and name work in <code>getBean()</code> .	Optional
scope	singleton (default) or prototype. Can also set request/session for web apps.	Optional (default: singleton)
primary	true/false — marks this bean as the primary candidate when multiple beans of same type exist.	Optional



5. Retrieving Beans from Context

Method	Example	When to use / Caveat
<code>getBean(id, Type.class)</code>	<code>context.getBean("orderService", OrderService.class)</code>	BEST — specify both id and type. No cast needed, no ambiguity.
<code>getBean(Type.class)</code>	<code>context.getBean(OrderService.class)</code>	Only works if exactly ONE bean of that type exists. Fails if two beans of same type.
<code>getBean(id)</code>	<code>context.getBean("orderService")</code>	Returns Object — requires manual cast. Avoid if possible.

```
// Best approach – id + type
OrderService order = context.getBean("orderService", OrderService.class);

// By type only – fails if two beans of same type exist
OrderService order = context.getBean(OrderService.class);
// NoUniqueBeanDefinitionException if cardPaymentService and upiPaymentService both exist

// By id only – requires cast
OrderService order = (OrderService) context.getBean("orderService");
```

6. Constructor Injection —

To inject a dependency via constructor, use `ref` inside the tag. Use `ref` for other beans, `value` for primitive/String values.



Field injection is NOT possible with XML-based configuration — private fields cannot be accessed from the XML file. Only constructor injection and setter injection are supported.

7. Setter Injection —

For setter injection, add a public setter method to the class, then use in the XML bean definition.

```
// OrderService.java – add a public setter
public class OrderService {
    private PaymentService paymentService;

    // public setter – Spring calls this after creating the object
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

The name attribute in maps to the setter method: name="paymentService" → Spring calls setPaymentService(). The method name must follow JavaBean convention (set + PascalCase of name).

8. Injecting Collections — List, Set, Map

Spring XML can inject Java collection types into constructors or setters.

```
Rohit  
Rohan
```

```
Aashutosh  
Rohit
```

9. Handling Multiple Beans of Same Type

If multiple beans implement the same interface, just reference the exact bean id you want — no `@Primary` or `@Qualifier` needed.

XML-based configuration avoids the `@Primary` / `@Qualifier` problem entirely by letting you explicitly name which bean to wire. This is one of the few advantages of XML.

10. Lifecycle Callbacks — init-method and destroy-method

Instead of `@PostConstruct` and `@PreDestroy` annotations, XML-based configuration uses `init-method` and `destroy-method` attributes on the tag.

```
// UserService.java — NO annotations, plain Java methods  
public class UserService {  
  
    public UserService() {  
        System.out.println("UserService created");  
    }  
  
    // This method will be called after creation + DI (like @PostConstruct)  
    public void init() {
```

```

        System.out.println("Post-construct phase");
    }

    // This method will be called before destruction (like @PreDestroy)
    public void cleanUp() {
        System.out.println("Pre-destroy phase");
    }
}

```

Prototype scope: destroy-method is NOT called even when context.close() is invoked. Spring does not manage the full lifecycle of prototype beans — garbage collection handles destruction.

11. Splitting XML into Multiple Files

Large projects split bean definitions across multiple XML files, then import them into a single root config file.

```

// Main.java – point to the root file only
ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("appConfig.xml");

```

12. Hybrid — XML + Annotation Together

Legacy projects often have XML-based config that is being migrated to annotations. During migration, both can coexist in one container.

```

// Main.java – still use ClassPathXmlApplicationContext
ClassPathXmlApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");
// Container now manages BOTH:
// - @Component-annotated classes (found via component-scan)
// - Explicitly declared tags in the XML

```

One container handles everything. Migration strategy: gradually move beans from XML to @Component annotations until the XML file is empty, then remove it.

13. XML vs Annotation — Full Comparison

Feature	XML-based	Annotation-based
Bean declaration		@Component on the class
Dependency injection	or	@Autowired on constructor/field/setter
Bean configuration file	beans.xml in resources folder	AppConfig.java with @Configuration
Component scanning		@ComponentScan on AppConfig
Lifecycle callbacks	init-method="init" destroy-method="cleanUp" on	@PostConstruct / @PreDestroy on methods
Scope	scope="prototype" on	@Scope("prototype") on the class
Multiple beans / ambiguity	Use explicit ref= to id — no ambiguity	@Primary or @Qualifier needed
Context class in Java	ClassPathXmlApplicationContext("beans.xml")	AnnotationConfigApplicationContext(AppConfig.class)
Field injection	NOT POSSIBLE — private fields inaccessible	Possible with @Autowired (uses Reflection API)
Verbosity	XML file grows large — harder to maintain	Clean Java code — annotations colocated with class
Use today?	Legacy codebases, migration projects	All new Spring and Spring Boot projects

Next Lecture: Spring Boot Introduction — Auto-configuration, Starters, Embedded Tomcat